



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе №3
по курсу «Защита информации»
на тему: «Алгоритм шифрования AES»
Вариант № 11

Студент ИУ7-71Б
(Группа)

(Подпись, дата)

Постнов С. А.
(Фамилия И. О.)

Преподаватель

(Подпись, дата)

Чиж И. С.
(Фамилия И. О.)

2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитический раздел	4
1.1 Алгоритм AES	4
1.1.1 Подготовка ключа	4
1.1.2 Начальный раунд	4
1.1.3 Основные раунды	4
1.2 PCBC	4
2 Конструкторская часть	6
2.1 Требования к программному обеспечению	6
2.2 Реализация алгоритмов	6
3 Технологический раздел	8
3.1 Средства реализации	8
3.2 Реализация алгоритмов	8
3.3 Тестирование	15
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17

ВВЕДЕНИЕ

Целью данной лабораторной работы является реализация в виде программы шифровального алгоритма AES в режиме работы PCBC.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) описать шифровальный алгоритм AES;
- 2) описать режим работы PCBC;
- 3) определить средства программной реализации;
- 4) реализовать программу, реализующую алгоритм шифрования AES;
- 5) протестировать реализованную программу.

1 Аналитический раздел

1.1 Алгоритм AES

AES (англ. *Advanced Encryption Standard*; также *Rijndael*) — симметричный алгоритм блочного шифрования (размер блока 128 бит, ключ 128/192/256 бит) [1].

Далее опишем основные этапы данного алгоритма.

1.1.1 Подготовка ключа

Начальные ключи расширяются для создания ключей раундов. Количество раундов зависит от длины ключа: 10 раундов для 128 бит, 12 для 192 бит и 14 для 256 бит.

1.1.2 Начальный раунд

AddRoundKey: каждый байт блока данных складывается по модулю 2 с соответствующим байтом раундового ключа.

1.1.3 Основные раунды

Для каждого раунда, кроме последнего [1]:

- 1) **SubBytes:** каждый байт блока заменяется на соответствующий байт из S-буквенной таблицы (замена);
- 2) **ShiftRows:** ряды блоков сдвигаются влево на разное количество байт;
- 3) **MixColumns:** каждая колонка блока обрабатывается, чтобы смешать данные;
- 4) **AddRoundKey:** выполняется XOR с новым раундовым ключом.

1.2 PCBC

Эрсам, Мейер, Смит и Такман изобрели режим работы цепочки блоков шифрования (CBC) в 1976 году. В режиме CBC каждый блок открытого текста перед шифрованием связывается с предыдущим блоком зашифрованного текста. Таким образом, каждый блок зашифрованного текста зависит от всех блоков открытого текста, обработанных до этого момента. Чтобы сделать

каждое сообщение уникальным, в первом блоке должен использоваться вектор инициализации.

Если первый блок имеет индекс 1, математическая формула для шифрования CBC выглядит так:

$$C_i = E_k(P_i \oplus C_{i-1}), C_0 = IV \quad (1.1)$$

Для дешифрования:

$$P_i = D_k(C_i) \oplus C_{i-1}, C_0 = IV \quad (1.2)$$

CBC является наиболее часто используемым режимом работы. Его основные недостатки заключаются в том, что шифрование является последовательным и что сообщение должно быть дополнено до размера, кратного размеру блока шифрования. Один из способов решения этой последней проблемы — метод, известный как кража зашифрованного текста. При расшифровке с неправильным IV первый блок открытого текста будет повреждён, но последующие блоки открытого текста будут корректными. Это связано с тем, что каждый блок шифруется по схеме XOR с зашифрованным текстом предыдущего блока, а не с открытым текстом, поэтому нет необходимости расшифровывать предыдущий блок перед использованием его в качестве IV для расшифровки текущего блока. Это означает, что блок открытого текста можно восстановить по двум соседним блокам зашифрованного текста. Следовательно, расшифровку можно выполнять параллельно.

PCBC был разработан таким образом, чтобы при расшифровке, как и при шифровании, небольшие изменения в зашифрованном тексте распространялись на весь текст. В режиме PCBC каждый блок открытого текста перед шифрованием складывается операцией XOR с предыдущим блоком открытого текста и предыдущим блоком зашифрованного текста. Как и в режиме CBC, в первом блоке используется вектор инициализации. В отличие от CBC, при расшифровке PCBC с неправильным IV (вектором инициализации) все блоки открытого текста будут повреждены [2].

2 Конструкторская часть

2.1 Требования к программному обеспечению

К программе предъявлен ряд требований:

- 1) шифрование произвольного файла или текстового сообщения;
- 2) возможность работы с пустым, однобайтовым файлом;
- 3) возможность обрабатывать файл архива.

2.2 Реализация алгоритмов

На рисунке 2.1 представлена схема алгоритма раунда AES.

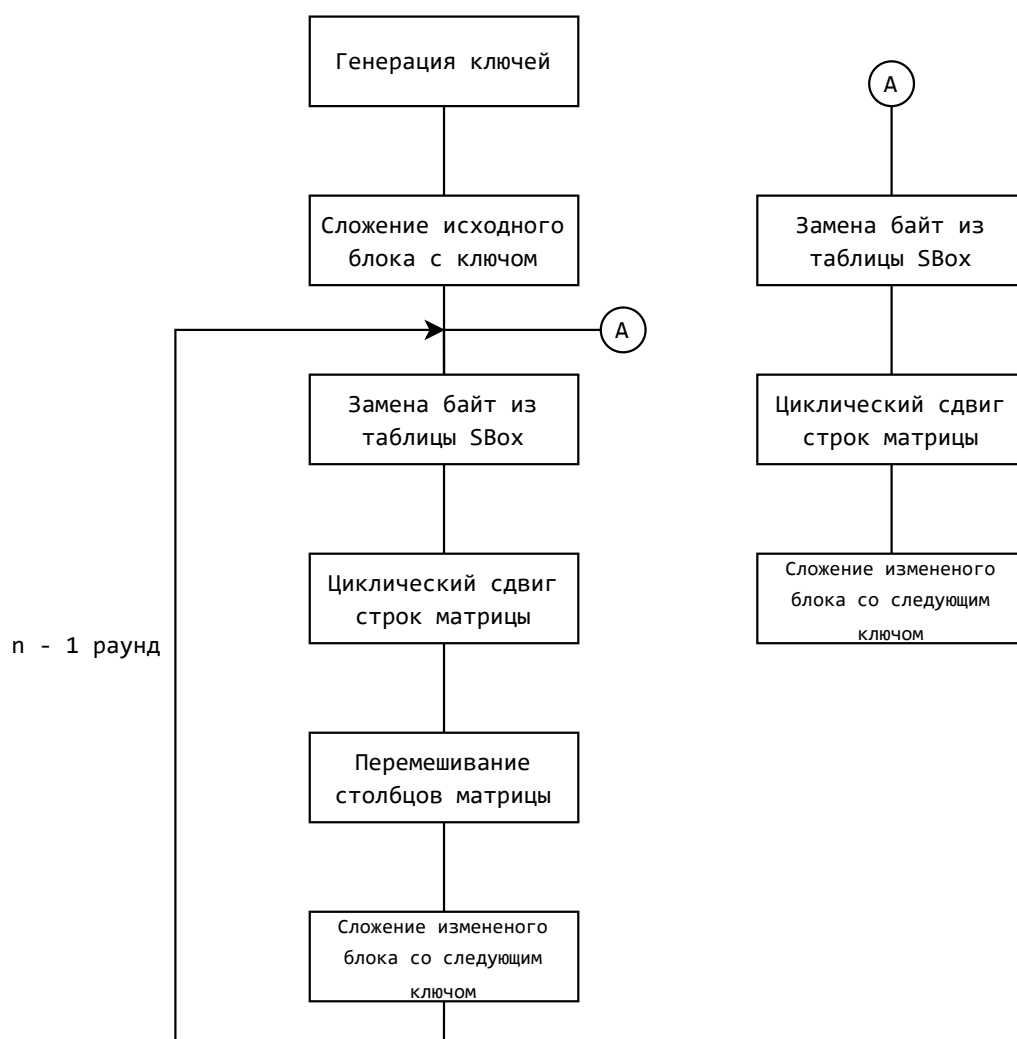


Рисунок 2.1 – Схема шифровального алгоритма AES

На рисунке 2.2 - 2.3 представлены схема шифрования и дешифрования в режиме PCBC.

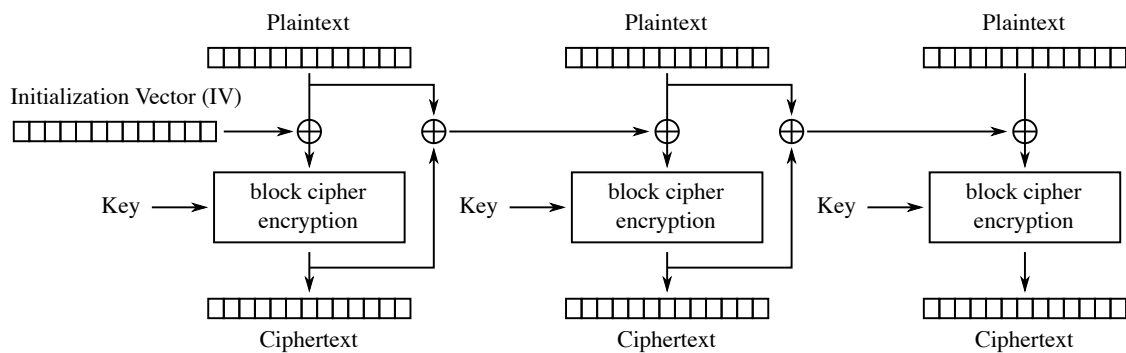


Рисунок 2.2 – Схема шифрования в режиме PCBC

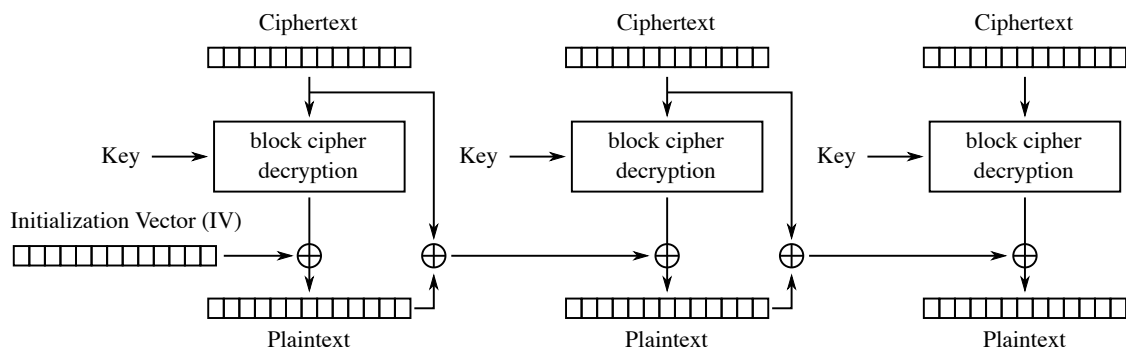


Рисунок 2.3 – Схема дешифрования в режиме PCBC

3 Технологический раздел

3.1 Средства реализации

В качестве языка программирования, используемого при написании данной лабораторной работы, был выбран C++ [3].

3.2 Реализация алгоритмов

В листингах 3.1 – 3.2 представлены реализации разрабатываемых модулей.

Листинг 3.1 – Реализация класса алгоритма AES

```
1  #include "aes.hpp"
2
3  block AES::EncryptBlock(block& block128, const key& key128) {
4      const auto keys = GetKeys128(key128);
5
6      auto state = ToMatrix(block128);
7
8      AddRoundKey(state, keys[0]);
9
10     for (int i = 1; i < 10; ++i) {
11         SubBytes(state);
12         ShiftRows(state);
13         MixColumns(state);
14         AddRoundKey(state, keys[i]);
15     }
16
17     SubBytes(state);
18     ShiftRows(state);
19     AddRoundKey(state, keys[10]);
20
21     return ToArray(state);
22 }
23
24 block AES::DecryptBlock(block& block128, const key& key128) {
25     const auto keys = GetKeys128(key128);
26
27     auto state = ToMatrix(block128);
28
29     AddRoundKey(state, keys[10]);
```



```

30
31     for (int i = 9; i > 0; --i) {
32         InvShiftRows(state);
33         InvSubBytes(state);
34         AddRoundKey(state, keys[i]);
35         InvMixColumns(state);
36     }
37
38     InvShiftRows(state);
39     InvSubBytes(state);
40     AddRoundKey(state, keys[0]);
41
42     return ToArray(state);
43 }
44
45 key AES::sub_word(const key &word) {
46     vector<uint8_t> result(word.size());
47
48     for (size_t i = 0; i < word.size(); ++i)
49         result[i] = S_BOX[word[i]];
50
51     return result;
52 }
53
54 key AES::rot_word(const key &word) {
55     vector<uint8_t> result = word;
56     const uint8_t temp = result[0];
57
58     for (size_t i = 0; i < word.size() - 1; ++i)
59         result[i] = result[i + 1];
60
61     result[word.size() - 1] = temp;
62
63     return result;
64 }
65
66 vector<key> AES::GetKeys128(key key128) {
67     vector roundKeys(11, key(16));
68
69     roundKeys[0] = std::move(key128);
70

```

```

71     for (int i = 1; i <= 10; ++i) {
72         key tempWord = roundKeys[i - 1];
73
74         tempWord = rot_word(tempWord);
75         tempWord = sub_word(tempWord);
76         tempWord[0] ^= R_CON[i - 1];
77
78         for (int j = 0; j < 16; ++j)
79             roundKeys[i][j] = roundKeys[i - 1][j] ^ tempWord[j];
80     }
81
82     return roundKeys;
83 }
84
85 void AES::AddRoundKey(matrix& block, const key& roundKey) {
86     for (int i = 0; i < block.size(); ++i)
87         for (int j = 0; j < block[0].size(); ++j)
88             block[i][j] ^= roundKey[i + j];
89 }
90
91 void AES::InvSubBytes(matrix &block) {
92     for (size_t i = 0; i < block.size(); ++i)
93         for (int j = 0; j < block[0].size(); ++j)
94             block[i][j] = S_BOX_INV[block[i][j]];
95 }
96
97 void AES::SubBytes(matrix &block) {
98     for (size_t i = 0; i < block.size(); ++i)
99         for (int j = 0; j < block[0].size(); ++j)
100             block[i][j] = S_BOX[block[i][j]];
101 }
102
103 matrix AES::ToMatrix(const block &block) {
104     matrix mtx(4, vector<uint8_t>(4));
105
106     for (int i = 0; i < 4; ++i)
107         for (int j = 0; j < 4; ++j)
108             mtx[j][i] = block[i * 4 + j];
109
110     return mtx;
111 }

```

```

112
113 block AES::ToArray(const matrix &mtx) {
114     block block128(16);
115
116     for (int i = 0; i < 4; ++i)
117         for (int j = 0; j < 4; ++j)
118             block128[(i * 4) + j] = mtx[j][i];
119
120     return block128;
121 }
122
123 void AES::ShiftRows(matrix &mtx) {
124     for (int i = 1; i < mtx.size(); ++i)
125         std::rotate(mtx[i].begin(), mtx[i].begin() + i,
126                     mtx[i].end());
127 }
128
129 void AES::InvShiftRows(matrix &mtx) {
130     for (int i = 1; i < mtx.size(); ++i)
131         std::rotate(mtx[i].rbegin(), mtx[i].rbegin() + i,
132                     mtx[i].rend());
133 }
134
135 void AES::MixColumns(matrix& mtx) {
136     for (size_t i = 0; i < 4; ++i) {
137         uint8_t s0 = mtx[0][i];
138         uint8_t s1 = mtx[1][i];
139         uint8_t s2 = mtx[2][i];
140         uint8_t s3 = mtx[3][i];
141
142         mtx[0][i] = GFMultiply(s0, 0x02) ^ GFMultiply(s1, 0x03)
143             ^ s2 ^ s3;
144         mtx[1][i] = s0 ^ GFMultiply(s1, 0x02) ^ GFMultiply(s2,
145             0x03) ^ s3;
146         mtx[2][i] = s0 ^ s1 ^ GFMultiply(s2, 0x02) ^
147             GFMultiply(s3, 0x03);
148         mtx[3][i] = GFMultiply(s0, 0x03) ^ s1 ^ s2 ^
149             GFMultiply(s3, 0x02);
150     }
151 }
152

```

```

147 void AES::InvMixColumns(matrix &mtx) {
148     for (size_t i = 0; i < 4; ++i) {
149         uint8_t s0 = mtx[0][i];
150         uint8_t s1 = mtx[1][i];
151         uint8_t s2 = mtx[2][i];
152         uint8_t s3 = mtx[3][i];
153
154         mtx[0][i] = GFMultiply(s0, 0x0E) ^ GFMultiply(s1, 0x0B)
            ^ GFMultiply(s2, 0x0D) ^ GFMultiply(s3, 0x09);
155         mtx[1][i] = GFMultiply(s0, 0x09) ^ GFMultiply(s1, 0x0E)
            ^ GFMultiply(s2, 0x0B) ^ GFMultiply(s3, 0x0D);
156         mtx[2][i] = GFMultiply(s0, 0x0D) ^ GFMultiply(s1, 0x09)
            ^ GFMultiply(s2, 0x0E) ^ GFMultiply(s3, 0x0B);
157         mtx[3][i] = GFMultiply(s0, 0x0B) ^ GFMultiply(s1, 0x0D)
            ^ GFMultiply(s2, 0x09) ^ GFMultiply(s3, 0x0E);
158     }
159 }
160
161 uint8_t AES::GFMultiply(uint8_t x, uint8_t y) {
162     uint8_t result = 0;
163     uint8_t hbit = 0;
164
165     for (int i = 0; i < 8; ++i) {
166         if (y & 1)
167             result ^= x;
168
169         hbit = x & 0x80;
170         x <<= 1;
171         if (hbit)
172             x ^= 0x1B;
173
174         y >>= 1;
175     }
176
177     return result;
178 }

```

Листинг 3.2 – Реализация класса шифровального алгоритма AES в режиме работы PCBC

```

1 #pragma once
2
3 #include <vector>

```

```

4 #include <stdexcept>
5
6 #include "aes.hpp"
7
8 class PCBC {
9 public:
10     std::vector<uint8_t> Encrypt(const std::vector<uint8_t>&
        plaintext, const block& iv, const key& key128) {
11         std::vector<uint8_t> ciphertext;
12         ciphertext.reserve(plaintext.size());
13
14         block prevCipherBlock = iv;
15         AES aes;
16
17         for (size_t i = 0; i < plaintext.size(); i += 16) {
18             block b(16);
19             const size_t blockSize = std::min<size_t>(16,
                plaintext.size() - i);
20             std::copy_n(plaintext.begin() + i, blockSize,
                b.begin());
21
22             for (size_t j = 0; j < 16; ++j)
23                 b[j] ^= prevCipherBlock[j];
24
25             auto cipherBlock = aes.EncryptBlock(b, key128);
26             ciphertext.insert(ciphertext.end(),
                cipherBlock.begin(), cipherBlock.end());
27
28             for (size_t j = 0; j < 16; ++j)
29                 prevCipherBlock[j] = plaintext[i + j] ^
                    cipherBlock[j];
30         }
31
32         return ciphertext;
33     }
34
35     std::vector<uint8_t> Decrypt(const std::vector<uint8_t>&
        ciphertext, const block& iv, const key& key128) {
36         std::vector<uint8_t> plaintext;
37         plaintext.reserve(ciphertext.size());
38

```

```

39     block prevCipherBlock = iv;
40     AES aes;
41
42     for (size_t i = 0; i < ciphertext.size(); i += 16) {
43         block cipherBlock(16);
44         std::copy_n(ciphertext.begin() + i, 16,
45                     cipherBlock.begin());
46
47         auto decryptedBlock = aes.DecryptBlock(cipherBlock,
48         key128);
49
50         for (size_t j = 0; j < 16; ++j)
51             decryptedBlock[j] ^= prevCipherBlock[j];
52
53         plaintext.insert(plaintext.end(),
54                         decryptedBlock.begin(), decryptedBlock.end());
55
56         for (size_t j = 0; j < 16; ++j)
57             prevCipherBlock[j] = decryptedBlock[j] ^
58             cipherBlock[j];
59     }
60
61     return plaintext;
62 }
63 };

```

3.3 Тестирование

В таблице 3.1 представлены функциональные тесты.

Таблица 3.1 – Функциональные тесты

№	Входные данные	Выходные данные
1	пустая строка	пустая строка
2	пустой файл	пустой файл
3	aaaaa	зашифрованный "aaaaa"
4	зашифрованный "aaaaa"	aaaaa
5	abcde	зашифрованный "abcde"
6	зашифрованный "abcde"	abcde
7	файл file.txt	зашифрованный файл file.txt
8	зашифрованный файл file.txt	файл file.txt
9	файл file.bmp	зашифрованный файл input.bmp
10	зашифрованный файл file.bmp	файл file.bmp
11	файл file.zip	зашифрованный файл file.zip
12	зашифрованный файл file.rar	файл file.zip

ЗАКЛЮЧЕНИЕ

Цель, поставленная в начале работы, была достигнута. Кроме того были достигнуты все поставленные задачи:

- 1) описан шифровальный алгоритм AES;
- 2) описан режим работы PCBC;
- 3) определены средства программной реализации;
- 4) реализована программа, реализующая алгоритм шифрования AES;
- 5) протестирована реализованная программу.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Как устроен AES. — [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/articles/112733/> (дата обращения: 30.10.2024).
2. Режим PCBC. — [Электронный ресурс]. — Режим доступа: https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation (дата обращения: 30.10.2024).
3. C++ language. — [Электронный ресурс]. — Режим доступа: <https://en.cppreference.com/w/cpp/language> (дата обращения: 30.10.2024).